CENTRO PAULA SOUZA
FACULDADE DE TECNOLOGIA DE RIBEIRÃO PRETO
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Glauber Noccioli de Souza

Juliana Maniglia Giaccomassi

Niele Dias Mendes

Rachel Carvalho Sanches Pereira

Rafaela Fernandes Amado

Projeto de software para gestão de cursos

Projeto de Desenvolvimento de Software

Ribeirão Preto
2024

SUMÁRIO

Sumário

1. CONTEXTUALIZAÇÃO 4
1.1 OBJETIVO 4
1.2 PÚBLICO-ALVO E PERSONAS 4
2. ANÁLISE DO NEGÓCIO 6
2.1 ANÁLISE DE VIABILIDADE 6
2.3 STAKEHOLDERS 7
2.4 REQUISITOS FUNCIONAIS 7
2.5 REQUISITOS NÃO-FUNCIONAIS 9
3. ANÁLISE DO SISTEMA 10
3.1 ARQUITETURA 10
3.2 DIAGRAMA DE CASO DE USO 11
3.3 PROTOTIPAÇÃO 19
3.4 TECNOLOGIAS PARA DESENVOLVIMENTO 24
4. PLANO DE EXECUÇÃO 24
Plano de Execução para Tela de Login e Controle de Acesso 24
1. Tarefas Específicas do Projeto 24
Plano de Execução para Tela de Gerenciamento Financeiro 27
Plano de Execução para Tela de Gerenciamento de Alunos 29
Plano de Execução para Tela de Frequência de instrutores 31
1. Tarefas Específicas do Projeto 31
1.1. Desenvolvimento 31
1.2. Testes 32
1.3. Implantação 32
Plano de Execução para Tela de Frequência de Alunos 32

1. Tarefas Específicas do Projeto 33
1.1. Desenvolvimento 33
1.2. Testes 33
1.3. Implantação 34
Plano de Execução para Tela de Cadastro de Alunos 34
Plano de Execução para Tela de Cadastro de Cursos 36
Plano de Execução para Tela Emissão de Certificados 37
1. Tarefas Específicas do Projeto 38
Plano de Execução para Tela Relatório de Aprovações 39
1. Tarefas Específicas do Projeto 40
Cronograma de Atividades 42
Glossário 42

1. CONTEXTUALIZAÇÃO

Empresas que oferecem cursos diversos tem muitas informações para gerir: alunos e seus dados, turmas com alunos matrÍculas e instrutores responsáveis, instrutores e suas competência e horários ocupados e livres, informações sobre os cursos, suas grades curriculares e critérios de aprovação, frequência dos alunos, certificados emitidos dentre outras.

Com vista a facilitar a operação de empresas desse segmento, este projeto se propõe a ter funcionalidades que auxiliem o trabalho de gestão e administração de uma empresa que oferece cursos diversos.

Por meio dele será possível cadastrar alunos, instrutores, cursos, turmas. Também será possível ver os horários já preenchidos e livres dos instrutores. O controle de frequência, por ser um requisito de aprovação dos alunos nos cursos também vai ser feito pelo software.

O aluno ao concluir o curso, recebe um certificado de conclusão do curso. Se ele for menor de idade, o certificado será encaminhado ao email do responsável.

Será possível cadastrar as turmas de cada curso, e associar o instrutor e os alunos que dela fazem parte.

Este software se propõe a ser funcional e ao mesmo tempo, ter uma interface convidativa e respeitando as melhores práticas sobre experiência de usuário.

1.1 OBJETIVO

Desenvolver uma plataforma online para gestão de empresas que oferecem cursos diversos.

1.2 PÚBLICO-ALVO E PERSONAS

- **Público-Alvo:** gestores, administradores, alunos e instrutores vinculados a empresas que oferecem cursos diversos.
    - ***Personas:***

	Nome: João Ferreira Idade: 42 Função: Responsável Administrativo			
Comportamentos e Hábitos	Metas e Objetivos	Desafios e Necessidades	Preferencias de Uso	Feedback e Opiniões

João é organizado e meticuloso, revisa relatórios diários e acompanha o desempenho da equipe.	Manter a escola financeiramente saudável e otimizar os processos administrativos.	Necessita de um sistema centralizado e enfrenta desafios na integração de novas tecnologias.	Prefere interfaces claras com gráficos e relatórios automáticos.	Acredita que o sistema deve economizar tempo e apresentar as informações de forma rápida.

	Nome: Maria da Silva Idade: 35 Função: Secretária Escolar			
Comportamentos e Hábitos	Metas e Objetivos	Desafios e Necessidades	Preferencias de Uso	Feedback e Opiniões
Comunicativa e organizada, tem o hábito de confirmar regularmente as informações de contato.	Facilitar o processo de matrícula e garantir a organização dos dados dos alunos.	Precisa de uma solução que automatize matrículas e emissão de certificados.	Prefere um sistema simples e rápido para emissão de certificados e notificações.	Valoriza um design limpo e fácil de navegar.

	Nome: André Lopes Idade: 30 Função: Instrutor de Programação			
Comportamentos e Hábitos	Metas e Objetivos	Desafios e Necessidades	Preferencias de Uso	Feedback e Opiniões
Dedicado e engajado, gosta de usar tecnologia para dinamizar suas aulas.	Ensinar de forma eficaz e acompanhar o progresso dos alunos.	Precisa de ferramentas que facilitem o controle de presença e inserção de notas.	Prefere um sistema acessível pelo celular para controle das turmas.	Acha que o sistema deve ser integrado com ferramentas educacionais.

	Nome: Fabiana Souza Idade: 38 Função: Coordenadora Pedagógica			
Comportamentos e Hábitos	Metas e Objetivos	Desafios e Necessidades	Preferencias de Uso	Feedback e Opiniões
Detalhista, revisa a agenda e o currículo dos cursos semanalmente.	Garantir a eficiência da escola sem sobrecarregar os professores.	Precisa de uma ferramenta clara para gerenciar horários e desempenho.	Prefere visão global das turmas e relatórios personalizados.	O sistema deve ser flexível e fácil de ajustar.

	Nome: Lucas Pereira Idade: 16 Função: Aluno de Programação			
Comportamentos e Hábitos	Metas e Objetivos	Desafios e Necessidades	Preferencias de Uso	Feedback e Opiniões
Curioso e dedicado, acessa o sistema regularmente para verificar seu progresso.	Passar no curso com boas notas e obter seu certificado.	Precisa de uma plataforma que acompanhe seu progresso e o notifique sobre prazos.	Prefere um sistema acessível pelo celular e com alertas de notas e faltas.	Valoriza um sistema rápido e direto, com visualização clara de informações.

2. ANÁLISE DO NEGÓCIO

2.1 ANÁLISE DE VIABILIDADE

Entende-se que o software é viável, pois existe uma boa fatia de mercado ocupada pelo segmento que será atendido. Muitas empresas do referido segmento ainda trabalham com papel ou planilha excel. Não existe um software de mercado

generalista para este fim. Geralmente cada empresa desenvolve um software para atender suas necessidades. Propõe-se aqui fazer um software com as boas práticas desse mercado, para que possa ser oferecido a diversas empresas que podem configurá-lo segundo suas necessidades. Tendo mais de um cliente potencial para o mesmo produto, o custo do desenvolvimento da plataforma pode ser dividido, ficando mais atrativo.

2.3 STAKEHOLDERS

Nome	Ocupação
Magno	Dono de uma empresa ofertante de cursos diversos.

2.4 REQUISITOS FUNCIONAIS

Os requisitos funcionais referem-se às especificações detalhadas das funções e operações que um sistema ou software deve realizar para atender às necessidades do usuário. Eles descrevem as funcionalidades específicas que o sistema deve fornecer, como recursos, capacidades e desempenho.

Esses requisitos geralmente delineiam as ações que o sistema deve realizar em resposta a entradas específicas, definindo assim o comportamento funcional esperado do sistema, se concentrando nas ações que o sistema deve executar para atender às necessidades do usuário. A Tabela 1 a seguir lista os requisitos funcionais do software especialmente criado para atender uma Escola de Cursos.

ID	**Requisito Funcional**	**Descrição**
RF01	Cadastrar usuário ‘Aluno’	Cadastrar os usuários ‘alunos’ no sistema, coletando os dados pessoais (Nome, Idade, RG, CPF, Endereço, e-mail, Telefone, profissão) e os dados dos responsáveis (no caso de o aluno ser menor de idade)
RF02	Cadastrar usuário ‘Instrutor’	Cadastrar os usuários ‘Instrutores’’ no sistema, coletando dados pessoais (Nome, Idade, RG, CPF, Endereço, e-mail, Telefone, Currículo) e os dias e horários disponíveis para assumir novas turmas.

RF03	Cadastrar usuário ‘Responsável Administrativo’	Cadastrar o Usuário responsável pela administração da escola, coletando os dados pessoais (Nome, Idade, RG, CPF, Endereço, e-mail, Telefone)
RF04	Cadastrar Cursos	O sistema permitirá o cadastro de cursos, incluindo a carga horária, nome do professor, plano de ensino e turmas (a iniciar ou em andamento). Esse recurso permitirá acessar a agenda dos professores, a fim de avaliar a disponibilidade de horários dos mesmos.
RF05	Liberar as permissões de usuários	Cada usuário terá permissões específicas liberadas, por exemplo: o responsável administrativo terá acesso a recursos administrativos e financeiros, os professores terão acesso aos perfis dos alunos, manipulando a lista de presença e subindo as notas, e os alunos terão somente o acesso à leitura de suas notas, presença e dos boletos em aberto)
RF06	Matricular “aluno’	Matricular o Aluno em um curso, ou mais de um.
RF07	Vincular Curso ao Instrutor	O sistema vinculará o professor ao curso, gerando automaticamente agenda sua agenda
RF08	Controlar a frequência dos alunos	O sistema poderá controlar a frequência dos alunos
RF09	Controlar a frequência dos Instrutores	O sistema poderá controlar a frequência dos Instrutores.
RF10	Gerenciar os perfis dos alunos	O sistema permitirá armazenar os dados dos alunos em seus perfis, onde constarão, além de seus dados pessoais, informações de notas,

		frequência, cursos em andamento ou finalizados e seus certificados já obtidos.
RF11	Gerar os Modelos dos certificados de conclusão do curso	O sistema gerará os certificados dos alunos matriculados em cada curso, baseados nos critérios: 70% de presença no curso, e nota mínima 7.
RF012	Gerenciamento das turmas	O professor terá acesso ao gerenciamento dos perfis das turmas, podendo subir as notas e frequência.
RF13	Gerenciamento financeiro	O sistema será capaz de controlar os pagamentos dos alunos, gerar boletos e notas fiscais. Além de indicar automaticamente a situação do aluno (adimplente ou inadimplente), possibilitando a gestão financeira.
RF14	Geração de Relatórios	O sistema terá a capacidade de gerar diversos relatórios, como frequência dos alunos, médias de notas, relatórios financeiros e análises estatísticas, como taxa de aprovação e de abandono do curso, proporcionando uma visão abrangente e detalhada para a tomada de decisões
RF15	Agenda	O sistema disponibilizará a agenda, com os dias e horários dos cursos, professores e turmas.

2.5 REQUISITOS NÃO-FUNCIONAIS

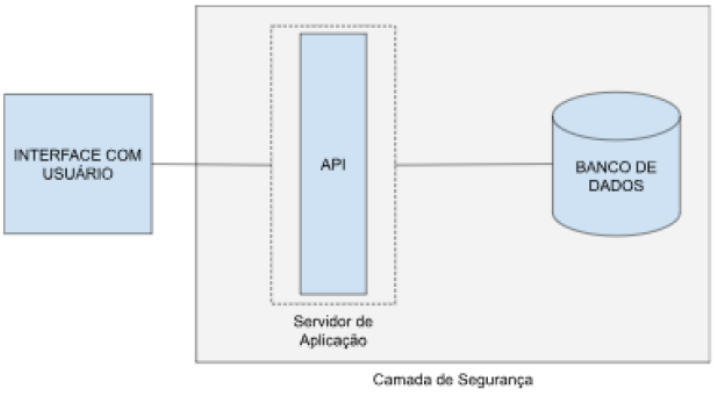
Diferentemente dos funcionais, os requisitos não funcionais concentram-se em características que não estão diretamente relacionadas às funcionalidades específicas do sistema, mas sim em atributos que afetam seu desempenho, confiabilidade, usabilidade e outros aspectos não diretamente ligados às operações explícitas do software. Isso inclui requisitos de desempenho, segurança, escalabilidade, usabilidade, confiabilidade e outros critérios de qualidade.

Esses requisitos não funcionais são críticos para garantir que o sistema seja eficiente, seguro e capaz de lidar com as demandas esperadas, indo além das funcionalidades específicas e abordando aspectos mais amplos. A Tabela 2 a seguir lista os requisitos não funcionais:

ID	Requisito Não-Funcional	Atributo
RNF01	O sistema deve ser fácil de usar e intuitivo.	Usabilidade
RNF02	O sistema deve responder corretamente às solicitações dos usuários.	Desempenho
RNF03	O sistema deve responder de forma imediata a cada clique ou comando do usuário.	Velocidade
RNF04	Atualizações e sincronizações instantâneas.	Desempenho
RNF05	Espaço de armazenamento suficiente	Desempenho
RNF06	Armazenamento em Nuvem	Confiabilidade
RNF07	Software online, hospedado em provedor com acesso 24/7.	Disponibilidade

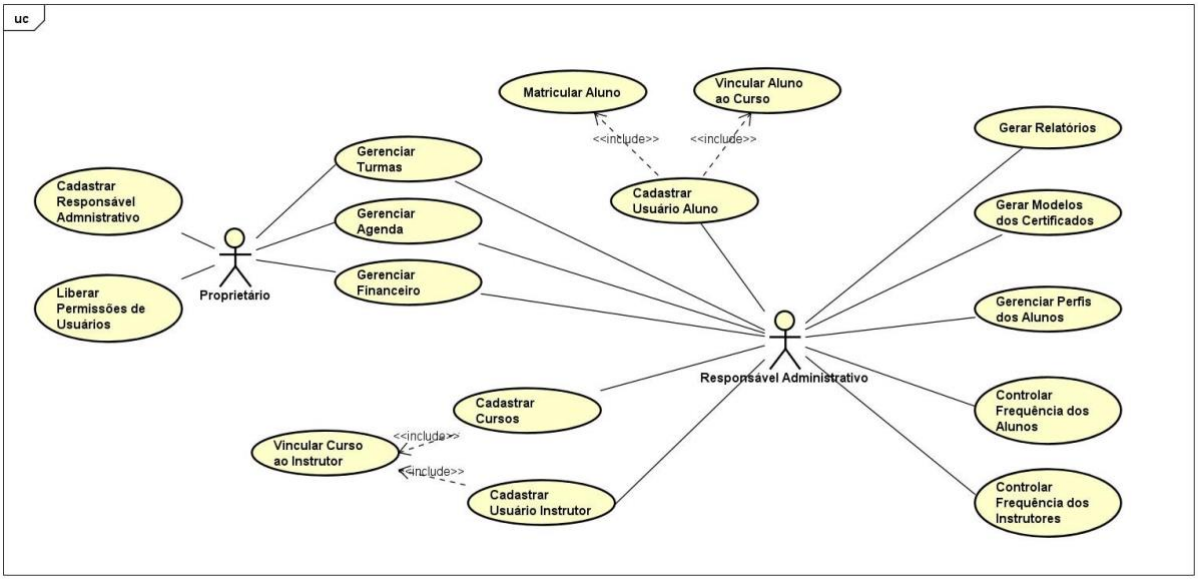
3. ANÁLISE DO SISTEMA

3.1 ARQUITETURA



- Interface com Usuário (Front-End): É a parte do aplicativo com a qual os usuários interagem diretamente. Ele é responsável pela apresentação visual e pela interação do usuário. A interface será desenvolvida na forma de um aplicativo multiplataforma.
- API (Back-End): A API atua como uma ponte entre o front-end e o banco de dados. Ela gerencia as solicitações do cliente, processa lógica de negócios e retorna dados relevantes. O objetivo da API é permitir que diferentes tipos de interfaces com usuário sejam integradas na plataforma. Assim, além de acesso por aplicativos, será possível (no futuro) acessar os recursos por meio de uma aplicação Web. Para uma clínica odontológica, a API deve lidar com operações como agendamento de consultas, acesso a informações do paciente e gerenciamento de profissionais.
- O banco de dados armazena e gerencia os dados do aplicativo. Para um aplicativo de clínica odontológica, os dados podem incluir informações sobre pacientes, profissionais da saúde, horários de consultas e históricos médicos.

3.2 DIAGRAMA DE CASO DE USO



Caso de Uso	**Cadastrar responsável administrativo**
Ator	Proprietário
Pré-Condições	1. O Proprietário já está logado ao sistema
Fluxo Principal	1. O proprietário acessa área especifica de cadastro de responsável administrativo;

	2. O proprietário cadastra um responsável administrativo fornecendo dados pessoais como: ○ Nome; ○ CPF; ○ RG; ○ Endereço; ○ Telefone; ○ Telefone alternativo; 3. O proprietário salva o cadastro.
Pós-Condições	1. O Auxiliar administrativo está cadastrado no sistema.
Fluxos Alternativos	1. O proprietário cancela o cadastro.

Caso de Uso	**Liberar Permissões de usuários**
Ator	Proprietário
Pré-Condições	1. O Proprietário já está logado ao sistema; 2. O Responsável administrativo ter cadastrado o usuário.
Fluxo Principal	1. O proprietário entra na aba de permissões de usuários; 2. O proprietário identifica o usuário ou grupo de usuários que deseja aplicar permissões; 3. O proprietário libera permissão ao usuário ou ao grupo de usuário. 4. O proprietário salva a nova configuração.
Pós-Condições	1. O usuário cadastrado está apto a acessar áreas específicas do software.
Fluxos Alternativos	1. O proprietário cancela as configurações que estava realizando.

Caso de Uso	**Gerenciar turmas**

Ator	Proprietário e Responsável Administrativo
Pré-Condições	1. O Ator já está logado ao sistema
Fluxo Principal	1. O ator acessa área especifica de turmas; 2. O ator tem a possibilidade de: ○ Criar e Deletar turmas; ○ Determinar turma para cada usuário; ○ Determinar instrutor para cada turma; ○ Determinar dias da semana da aula de cada turma; ○ Determinar o horário da aula de cada turma (Inicio, fim e duração) ○ Determinar o início e o fim da turma; 3. O ator salva as alterações.
Pós-Condições	1. Cada turma tem a sua data de início e fim, tempo de aula e tempo, alunos e instrutores.
Fluxos Alternativos	1. O ator cancela as alterações.

Caso de Uso	**Gerenciar Agenda**
Ator	Proprietário e Responsável Administrativo
Pré-Condições	1. O Ator já está logado ao sistema
Fluxo Principal	1. O ator acessa área de agendas; 2. O ator tem a possibilidade de: ○ Marcar na agenda início e fim de aula de cada turma; ○ Período de aula de cada turma; ○ Disponibilidade de funcionários; 3. O ator salva as alterações.

Pós-Condições	1. Agenda organizada e com os devidos filtros para facilidade de visualização
Fluxos Alternativos	1. O ator cancela as alterações.

Caso de Uso	**Gerenciar Financeiro**
Ator	Proprietário e Responsável Administrativo
Pré-Condições	1. O Ator já está logado ao sistema
Fluxo Principal	1. O ator acessa área do financeiro; 2. O ator tem a possibilidade de: ○ Calcular entrada de caixa; ○ Calcular a saída de caixa; ○ Verificar o total de saída; ○ Verificar o total de entrada; ○ Controlar notas fiscais; ○ Emitir boletos; ○ Calcular balanço mensal. 3. O ator salva as alterações.
Pós-Condições	1. Área financeira organizada
Fluxos Alternativos	1. O ator cancela as alterações.

Caso de Uso	**Cadastrar Usuário Aluno**
Ator	Responsável administrativo
Pré-Condições	1. O Responsável Administrativo deve estar cadastrado no sistema. 2. O Responsável Administrativo já está logado ao sistema

	3. O Responsável Administrativo já tenha cadastrado o curso no sistema
Fluxo Principal	1. O Responsável Administrativo acessa área especifica de cadastro de usuário aluno; 2. O Responsável Administrativo cadastra um usuário aluno com os seguintes nomes: ○ Nome; ○ CPF; ○ RG; ○ Endereço; ○ Telefone; ○ Nome do responsável; ○ CPF do responsável; ○ RG do responsável; ○ Endereço do responsável; ○ Telefone do responsável; 3. O Responsável Administrativo salva o cadastro.
Pós-Condições	1. O Auxiliar Administrativo tem que realizar a matricula do aluno quando o mesmo é inserido ao sistema. 2. O Auxiliar Administrativo tem que vincular o aluno a um curso.
Fluxos Alternativos	1. O Auxiliar Administrativo cancela o cadastro.

Caso de Uso	**Cadastrar Curso**
Ator	Responsável administrativo
Pré-Condições	1. O Responsável Administrativo deve estar cadastrado no sistema. 2. O Responsável Administrativo já está logado ao sistema

Fluxo Principal	1. O Responsável Administrativo acessa área especifica de cadastro de curso; 2. O Responsável Administrativo cadastra curso com os seguintes nomes: ○ Nome do curso; ○ Tamanho máximo da turma; ○ Tamanho Mínimo da turma; ○ Instrutores aptos a ministrarem o curso.
Pós-Condições	1. Curso está criado; 2. Possíveis instrutores indicados.
Fluxos Alternativos	1. O Auxiliar Administrativo cancela o cadastro.

Caso de Uso	**Cadastrar usuário instrutor**
Ator	Responsável Administrativo
Pré-Condições	1. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	1. O Responsável Administrativo acessa área especifica de cadastro de usuário instrutor; 2. O Responsável Administrativo cadastra um usuário instrutor fornecendo dados pessoais como: ○ Nome; ○ CPF; ○ RG; ○ Endereço; ○ Telefone; ○ Telefone alternativo; ○ Especialidade; ○ Disponibilidade. ○ Vincular o usuário instrutor a possíveis cursos. 3. O Responsável Administrativo salva o cadastro.

Pós-Condições	1. O Auxiliar administrativo está cadastrado no sistema.
Fluxos Alternativos	1. O Responsável Administrativo cancela o cadastro.

Caso de Uso	**Gerenciar Relatórios**
Ator	Responsável Administrativo
Pré-Condições	56. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	57. O Responsável Administrativo acessa área específica de relatórios; 58. O Responsável Administrativo filtra as colunas e as variáveis necessárias;
Pós-Condições	59. O Responsável Administrativo pode salvar o relatório em formato PDF ou CSV.
Fluxos Alternativos	60. O Responsável Administrativo não gera o relatório ou não o salva.

Caso de Uso	**Gerar Modelos dos Certificados**
Ator	Responsável Administrativo
Pré-Condições	61. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	62. O Responsável Administrativo acessa área específica de geração de modelos dos certificados; 63. O Responsável Administrativo escolhe o curso e a turma para o qual o certificado será gerado; 64. Deseleciona os alunos dessa turma que não receberão o certificado; 65. Os seguintes dados são carregados automaticamente em relatórios individuais: 11. Nome do aluno; 12. CPF; 13. RG; 14. Curso; 15. Tempo total de curso; 16. Assinatura digitalizada do proprietário do curso; 66. O Responsável Administrativo salva os documentos gerados.
Pós-Condições	67. O Auxiliar administrativo tem a opção de enviar os certificados automaticamente para os e-mails cadastrados.
Fluxos Alternativos	68. O Responsável Administrativo cancela a geração dos certificados.

Caso de Uso	Gerenciar Perfis dos Alunos
Ator	Responsável Administrativo
Pré-Condições	69. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	70. O Responsável Administrativo acessa área específica de usuários cadastrados; 71. O Responsável Administrativo pode ativar e desativar perfil dos alunos;
Pós-Condições	72. Os perfis de alunos estarão ativados ou desativados.
Fluxos Alternativos	73. Sem fluxo alternativo.

Caso de Uso	Controlar Frequência dos Alunos
Ator	Responsável Administrativo
Pré-Condições	74. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	75. O Responsável Administrativo acessa área específica das turmas; 76. Cadastrar atestados médicos em justificativas de faltas; 77. Acompanhar a frequência dos alunos.
Pós-Condições	78. Em caso de baixa frequência, o Responsável Administrativo entra em contato com o aluno ou responsável para entender motivos; 79. Alunos reprovados por não atingirem a frequência mínima.
Fluxos Alternativos	80. Sem fluxo alternativo

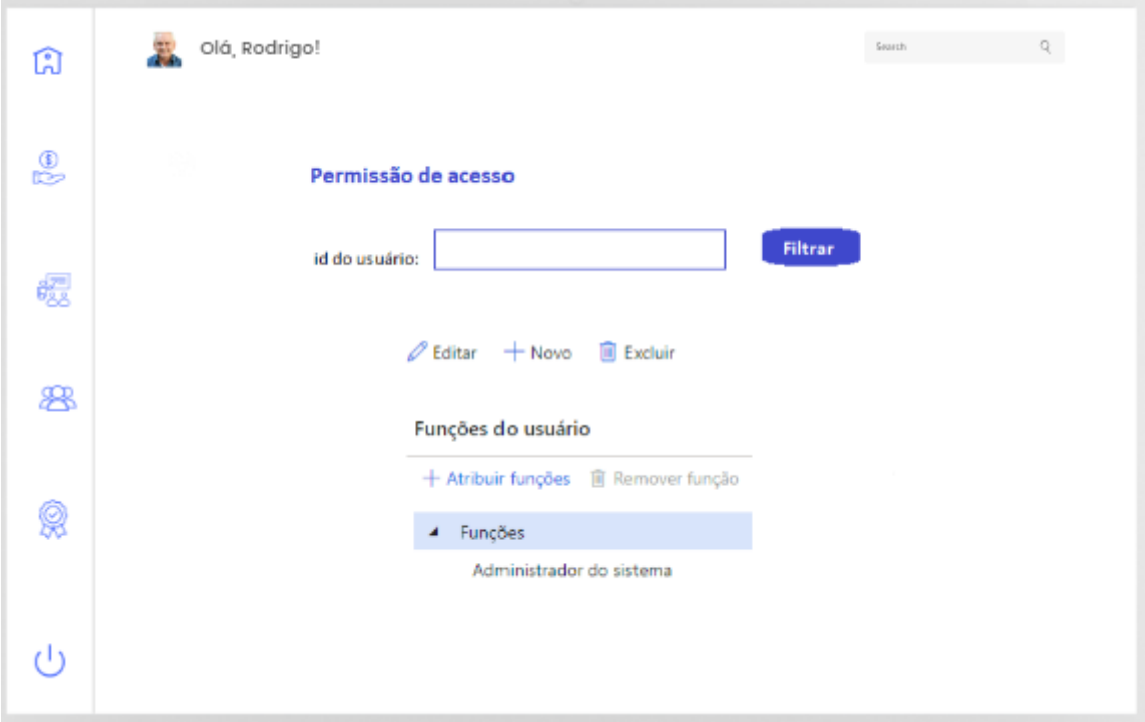
Caso de Uso	Controlar Frequência dos Instrutores
Ator	Responsável Administrativo
Pré-Condições	81. O Responsável Administrativo já está logado ao sistema
Fluxo Principal	82. O Responsável Administrativo acessa área específica dos instrutores; 83. Cadastrar atestados médicos em justificativas de faltas; 84. Alocar outro instrutor para a turma em caso de ausência;
Pós-Condições	85. Alocação de outro instrutor para a aula; 86. Repassar conteúdo programático para o instrutor substituto.
Fluxos Alternativos	87. Sem fluxo alternativo

3.3 PROTOTIPAÇÃO

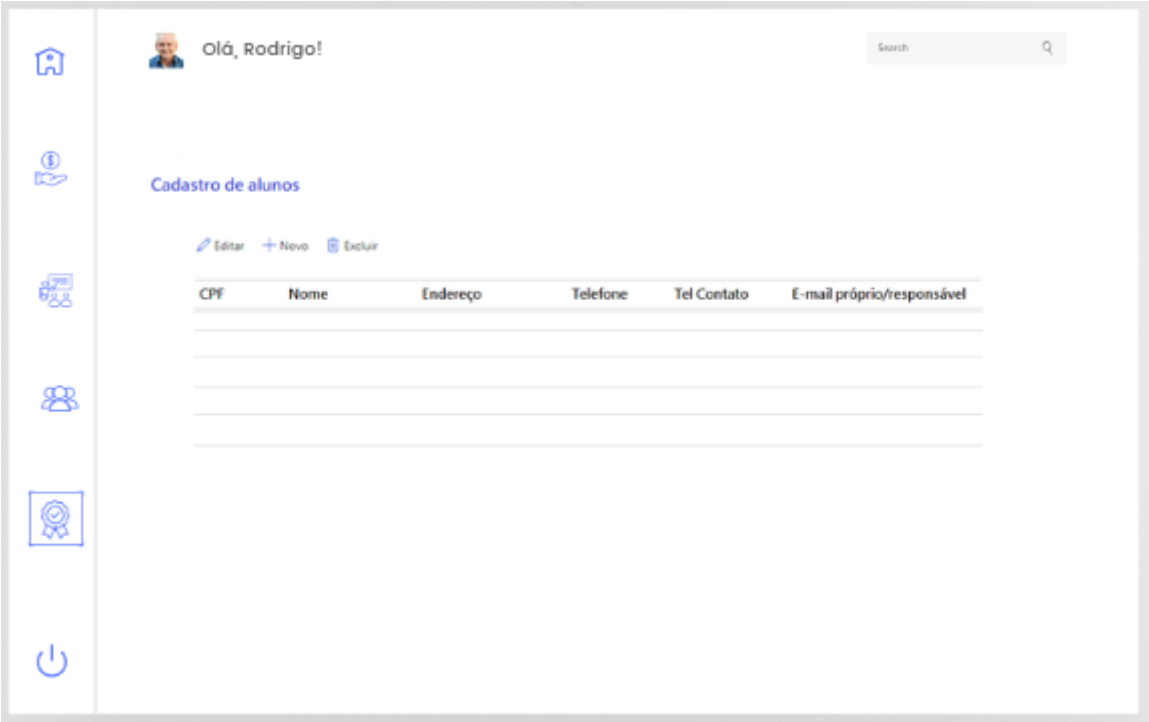
A prototipação de tela em engenharia de software é uma prática crucial no desenvolvimento de software que visa criar representações visuais das interfaces do usuário antes da implementação completa do sistema. Esses protótipos são versões simplificadas das interfaces que permitem aos desenvolvedores, designers e stakeholders experimentar e avaliar a aparência e a usabilidade de um software em estágios iniciais do processo de desenvolvimento. A prototipação de tela ajuda a esclarecer requisitos de design, funcionalidades e fluxos de interação, proporcionando uma compreensão mais tangível do produto final.
Abaixo são apresentados dez protótipos de telas para a plataforma computacional proposta neste trabalho.

Tela de login: Nela, o usuário terá onde inserir seu email e senha para acessar a plataforma e também recurso para cancelar se desejar não mais logar e para redefinir senha caso tenha esquecido.

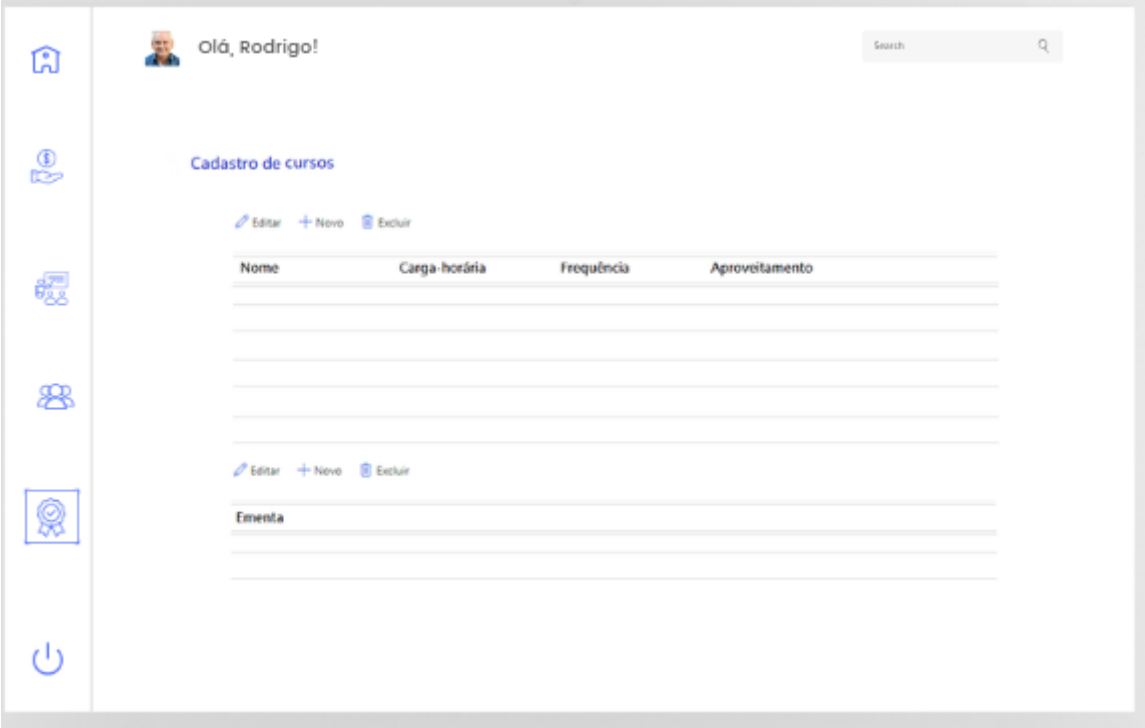
Tela de permissão de acesso: Nela, o administrador pode atribuir ou excluir funções a cada usuário do sistema.



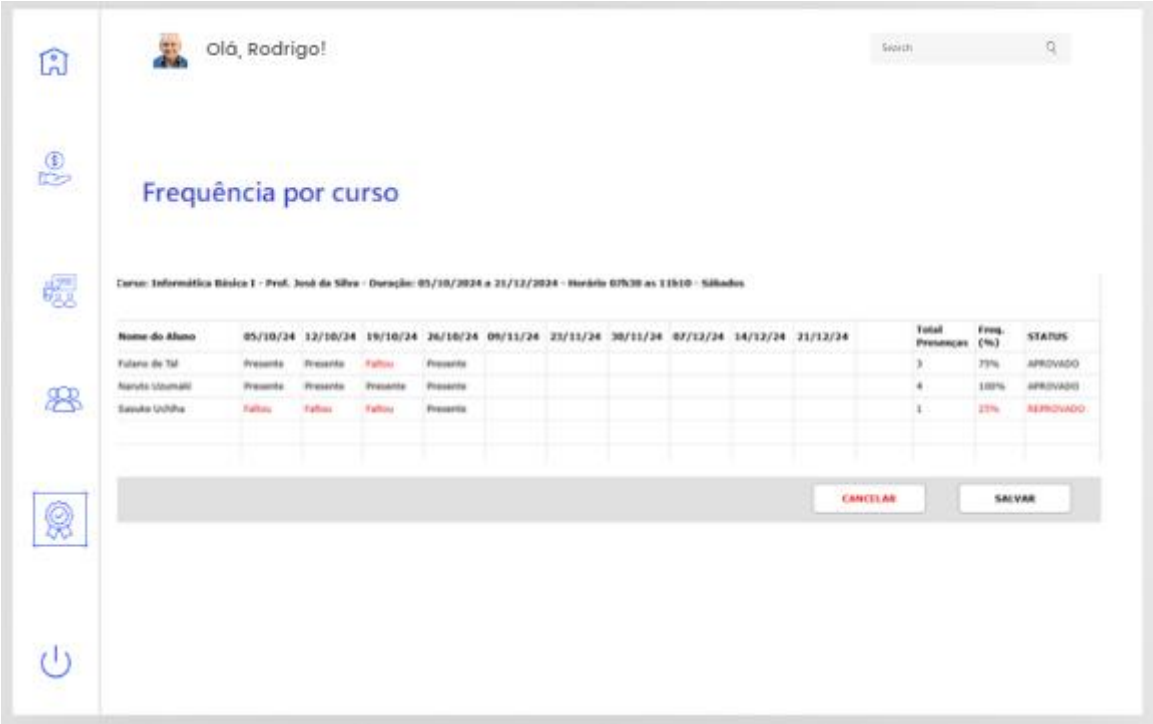
Tela de cadastro de alunos: nela é possível inserir, excluir e editar registro de novos alunos.



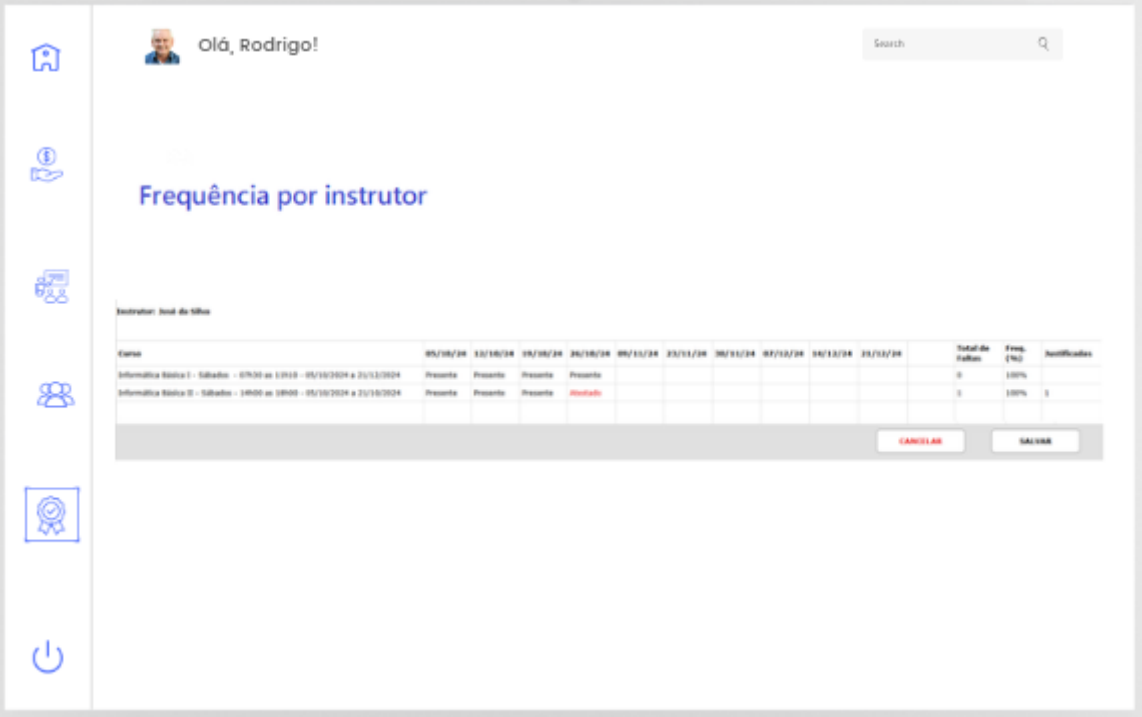
Tela de cadastro de cursos: nela é possível inserir, excluir e editar registro de novos cursos.



Tela de frequência por curso: nela é possível registra presença de cada aluno de determinado curso por dia de aula ministrada.



Tela de frequência por instrutor: nesta tela é possível colocar presença por dia por curso ministrado por cada instrutor.



Tela de gerenciamento financeiro: por meio de um dashboard é possível ver quantos cursos ativos, quantidade de alunos matriculados, instrutores responsáveis pela turma, faturamento e custos.

Tela de gestão de turma: podendo consultar frequência de cada aluno, saber se há algum aluno inadimplente e solicitar geração de certificado.

Tela de emissão de certificado: nesta tela, com o certificado gerado (ao se clicar na tela anterior), pode-se assiná-lo digitalmente, imprimir, exportar em pdf e sair da tela.

Tela de relatório de aprovações: pode-se filtrar por ano e semestre e visualizar todas as turmas do período. Por turma, é possível ver quais alunos foram aprovados. Também é possível imprimir esse relatório, além de visualizá-lo em tela.

3.4 TECNOLOGIAS PARA DESENVOLVIMENTO

A princípio a plataforma será disponibilizada em uma versão web, que utilizar-se-á das linguagens html com java script.

4. PLANO DE EXECUÇÃO

Plano de Execução para Tela de Login e Controle de Acesso

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados**
  - Criar tabelas para armazenamento de usuários, permissões e logs de acesso.
  - Definir relacionamentos entre usuários e perfis de acesso.
    Prazo: 1 dia

Recursos Necessários: Desenvolvedor backend, analista de banco de dados.

- **Desenvolver API de autenticação**
  - Criar endpoints para login e logout.
  - Implementar geração de tokens (JWT ou outra estratégia).
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend.
- **Criar interface de login**
  - Desenvolver uma tela de entrada com campos de usuário, senha e botão de acesso.
  - Adicionar funcionalidade de "Esqueceu a senha?".
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor frontend, designer UI/UX.
- **Implementar controle de acesso**
  - Configurar middleware para verificar permissões com base nos tokens.
  - Mapear permissões para diferentes níveis de usuário.
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend.

1.2. Testes

- **Realizar testes unitários**
  - Verificar validação de login, geração de token e proteção de rotas.
    Prazo: 1 dia
    Recursos Necessários: QA, desenvolvedor.
- **Executar testes de integração**
  - Garantir que a comunicação entre frontend, backend e banco de dados funcione.
    Prazo: 1 dia
    Recursos Necessários: QA.
- **Realizar testes de usabilidade**
  - Validar a experiência do usuário com a interface de login.
    Prazo: 1 dia
    Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**

  - Configurar servidores e bancos de dados para a solução.
    Prazo: 1 dia
    Recursos Necessários: DevOps.

- **Implantar sistema**
  - Subir a aplicação em produção.
  - Realizar smoke tests.
    Prazo: 1 dia
    Recursos Necessários: DevOps, desenvolvedor.

---

2. Dependências Entre Tarefas

1. Planejar estrutura do banco de dados → Pré-requisito para desenvolver API de autenticação e controle de acesso.
2. Desenvolver API de autenticação → Pré-requisito para realizar testes de integração.
3. Criar interface de login → Pré-requisito para testes de usabilidade.
4. Configurar ambiente de produção → Pré-requisito para implantação.

---

3. Marcos Importantes (Milestones)

1. Conclusão do desenvolvimento da API e interface → **Semana 1**.
2. Testes de integração e usabilidade finalizados → **Semana 2**.
3. Implantação em produção → **Semana 3**.

---

4. Monitoramento e Controle

4.1. Metodologia

- **Scrum**: Utilizar sprints semanais para entrega incremental.
- **Kanban**: Acompanhar o fluxo de tarefas no desenvolvimento e testes.

4.2. Ferramentas

- **Trello**: Gerenciamento das tarefas e prazos.
- **GitHub**: Controle de versão e colaboração.
- **Slack**: Comunicação da equipe.
- **Jenkins**: Automatizar testes e deploys.

4.3. Indicadores

- Percentual de tarefas concluídas por sprint.

- Tempo médio para resolução de bugs.
- Aderência aos prazos estabelecidos.

---

5. Identificação e Mitigação de Riscos

- **Risco**: Atraso na configuração do ambiente de produção.
  Mitigação: Configurar o ambiente de homologação antecipadamente como modelo.
- **Risco**: Bugs críticos durante testes de integração.
  Mitigação: Priorizar testes unitários rigorosos para evitar falhas críticas.

---

6. Entregáveis

1. Tela de login funcional e integrada.
2. Sistema de controle de acesso em produção.
3. Relatórios de logs de acesso gerados no banco de dados.

Plano de Execução para Tela de Gerenciamento Financeiro

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados**
  - Criar tabelas para transações financeiras (receitas e despesas), categorias, métodos de pagamento e histórico de movimentações.
  - Definir relacionamentos entre alunos, pagamentos, categorias e status financeiro.
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend, analista de banco de dados.
- **Desenvolver API para gerenciamento financeiro**
  - Criar endpoints para registro, edição, consulta e exclusão de transações financeiras.
  - Implementar cálculo automático de saldo e geração de relatórios financeiros.
    Prazo: 3 dias
    Recursos Necessários: Desenvolvedor backend.

- **Criar interface de gerenciamento financeiro**
  - Desenvolver tela para listar transações com filtros (por data, categoria, método de pagamento), exibir dados sobre turmas, adimplência dos alunos, instrutores.
  - Adicionar funcionalidade para registrar novas receitas ou despesas. Com os botões: Contas a pagar, Gerer Boleto, Cadastrar Conta, Receber Mensalidade, Fluxo de Caixa, Relatórios
  - Exibir meta do próximo mês e dados de faturamento, lucro e custos.
  - Implementar gráficos para visualização de dados financeiros (receitas, despesas e saldo) e matrículas de novos alunos.
    Prazo: 3 dias
    Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

- **Implementar relatórios financeiros**
  - Configurar exportação de relatórios em formatos PDF e Excel.
  - Adicionar análise de receitas e despesas por períodos (mensal, anual).
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend e frontend.

1.2. Testes

- **Testes unitários**
  - Validar endpoints financeiros, cálculos de saldo e geração de relatórios.
    Prazo: 1 dia
    Recursos Necessários: QA, desenvolvedor.
- **Testes de integração**
  - Garantir comunicação fluida entre o frontend, backend e banco de dados.
    Prazo: 1 dia
    Recursos Necessários: QA.
- **Testes de usabilidade**
  - Validar a experiência do usuário ao registrar transações e consultar dados financeiros.
    Prazo: 1 dia
    Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema financeiro.
    Prazo: 1 dia
    Recursos Necessários: DevOps.

- **Implantar sistema**

- Subir o sistema em produção e realizar smoke tests.
  Prazo: 1 dia
  Recursos Necessários: DevOps, desenvolvedor.

Plano de Execução para Tela de Gerenciamento de Alunos

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados**
  - Criar tabelas para alunos, turmas, e status de matrícula.
  - Definir relacionamentos entre alunos e turmas, incluindo histórico de alterações de status.
    Prazo: 1 dia
    Recursos Necessários: Desenvolvedor backend, analista de banco de dados

- **Desenvolver API para gerenciamento de alunos**
  - Criar endpoints para registrar, editar, consultar e excluir informações de alunos.
  - Adicionar lógica para vincular alunos a turmas e cursos.
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend.

- **Criar interface de gerenciamento de alunos**
  - Desenvolver uma tela para listar turmas com busca por curso, período e dia da semana.
  - Adicionar funcionalidade para vincular alunos a turmas e cursos.
  - Adicionar funcionalidade para exibir e editar presença, notas, frequência, status financeiro, bloquear aluno, gerar certificado.
  - Inserir os botões: salvar, imprimir, exportar e sair.
  - Implementar exportação de dados de alunos (PDF ou Excel).
    Prazo: 3 dias
    Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

- **Implementar relatórios acadêmicos**

- Configurar relatórios com informações sobre alunos e turmas, incluindo status de matrícula.
- Adicionar visualização de históricos de alteração de status dos alunos.
  Prazo: 2 dias
  Recursos Necessários: Desenvolvedor backend e frontend.

1.2. Testes

- **Testes unitários**
  - Validar endpoints de gerenciamento de alunos e exportação de relatórios.
    Prazo: 1 dia
    Recursos Necessários: QA, desenvolvedor.

- **Testes de integração**
  - Testar a interação entre backend, banco de dados e frontend.
    Prazo: 1 dia
    Recursos Necessários: QA.

- **Testes de usabilidade**
  - Validar a experiência do usuário ao gerenciar alunos e visualizar relatórios.
    Prazo: 1 dia
    Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema acadêmico.
    Prazo: 1 dia
    Recursos Necessários: DevOps.

- **Implantar sistema**
  - Subir o sistema em produção e realizar smoke tests.
    Prazo: 1 dia
    Recursos Necessários: DevOps, desenvolvedor.

Plano de Execução para Tela de Frequência de instrutores

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

Planejar a estrutura da base de dados para frequência

- **Criar tabelas para registrar a frequência dos instrutores, incluindo informações como data, horário, status de presença (presente, ausente, atrasado) e observações.**
- **Relacionar as tabelas de frequência com as tabelas de instrutores e turmas existentes.**
- **Prazo: 2 dias**
- **Recursos Necessários: Desenvolvedor backend, analista de banco de dados.**

- **Desenvolver API para gerenciamento da frequência**

Criar endpoints para registrar, editar e consultar frequência dos instrutores.

Implementar cálculo de métricas, como porcentagem de presença por período e horários mais frequentes de faltas.

Prazo: 3 dias

Recursos Necessários: Desenvolvedor backend.

- **Criar interface para registro e consulta de frequência**
  - **Desenvolver uma tela para exibir a lista de instrutores com filtros (por data, turma, status de presença).**
  - **Adicionar funcionalidade para registrar manualmente a frequência dos instrutores.**
  - **Exibir estatísticas, como gráficos de presença por período e notificações de ausência recorrente.**
  - **Prazo: 3 dias**
  - **Recursos Necessários: Desenvolvedor frontend, designer UI/UX.**

- **Implementar relatórios de frequência**
  - **Configurar exportação de relatórios de frequência em formatos PDF e Excel.**
  - **Adicionar análise detalhada por instrutor, turma e período.**
  - **Prazo: 2 dias**
  - **Recursos Necessários: Desenvolvedor backend e frontend.**

1.2. Testes

- **Testes unitários**
  - **Validar endpoints de registro, consulta e edição de frequência.**
  - **Prazo: 1 dia**
  - **Recursos Necessários: QA, desenvolvedor.**

- **Testes de integração**
  - **Garantir que o frontend, backend e banco de dados funcionem de maneira integrada.**
  - **Prazo: 1 dia**
  - **Recursos Necessários: QA.**

- **Testes de usabilidade**
  - **Validar a experiência do usuário ao registrar e consultar a frequência dos instrutores.**
  - **Prazo: 1 dia**
  - **Recursos Necessários: QA, designer UI/UX.**

1.3. Implantação

- **Configurar ambiente de produção**
  - **Preparar servidores e banco de dados para o sistema de frequência.**
  - **Prazo: 1 dia**
  - **Recursos Necessários: DevOps.**

- **Implantar sistema**
  - **Subir o sistema de frequência em produção e realizar smoke tests.**
  - **Prazo: 1 dia**
  - **Recursos Necessários: DevOps, desenvolvedor.**

Plano de Execução para Tela de Frequência de Alunos

---

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar a estrutura da base de dados para frequência de alunos**
  - Descrição:
  - Criar tabelas para registrar frequência dos alunos, incluindo informações como data, horário, status de presença (presente, ausente, atrasado) e observações.
  - Relacionar as tabelas de frequência com as tabelas de alunos, turmas e disciplinas.
  - Prazo: 2 dias
  - Recursos Necessários: Desenvolvedor backend, analista de banco de dados.

- **Desenvolver API para gerenciamento da frequência de alunos**
  - Descrição:
  - Criar endpoints para registrar, editar e consultar frequência dos alunos.
  - Implementar cálculo de métricas, como taxa de presença por aluno, turma e disciplina.
  - Prazo: 3 dias
  - Recursos Necessários: Desenvolvedor backend.

- **Criar interface para registro e consulta de frequência de alunos**
  - Desenvolver uma tela para exibir a lista de alunos com filtros (por data, turma, disciplina, status de presença).
  - Adicionar funcionalidade para registrar a frequência de maneira manual ou por integração com sistemas de leitura (ex.: QR Code ou biometria).
  - Exibir estatísticas visuais, como gráficos de presença por turma, disciplina e períodos específicos.
  - Prazo: 4 dias
  - Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

- **Implementar relatórios de frequência de alunos**
  - Configurar exportação de relatórios de frequência em formatos PDF e Excel.
  - Adicionar análise detalhada por aluno, turma, disciplina e período.
  - Prazo: 2 dias
  - Recursos Necessários: Desenvolvedor backend e frontend.

1.2. Testes

- **Testes unitários**

  - Descrição:
  - Validar endpoints de registro, consulta e edição de frequência dos alunos.
  - Prazo: 1 dia
  - Recursos Necessários: QA, desenvolvedor.

- **Testes de integração**
  - Descrição:
  - Garantir que o frontend, backend e banco de dados funcionem de maneira integrada.
  - Prazo: 1 dia
  - Recursos Necessários: QA.

- **Testes de usabilidade**
  - Descrição:
  - Validar a experiência do usuário ao registrar e consultar a frequência dos alunos.
  - Prazo: 1 dia
  - Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Descrição:
  - Preparar servidores e banco de dados para o sistema de frequência de alunos.
  - Prazo: 1 dia
  - Recursos Necessários: DevOps.
- **Implantar sistema**
  - Descrição:
  - Subir o sistema de frequência em produção e realizar smoke tests.
  - Prazo: 1 dia
  - Recursos Necessários: DevOps, desenvolvedor.

Plano de Execução para Tela de Cadastro de Alunos

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados**
  - Criar tabela para aluno
  - Definir relacionamentos entre alunos e turmas, incluindo histórico de alterações de status.
    Prazo: 2 dia
    Recursos Necessários: Desenvolvedor backend, analista de banco de dados

- **Desenvolver API para gerenciamento de alunos**
  - Criar endpoints para registrar, editar, consultar e excluir informações de alunos.
  - Adicionar lógica para vincular alunos a turmas e cursos.
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend.

- **Criar interface de cadastro de alunos**
  - Desenvolver uma tela para cadastrar os alunos
  - Adicionar funcionalidade para vincular alunos a turmas e cursos.
  - Adicionar funcionalidade para novo, editar e excluir.
  - Inserir os botões: salvar e sair.
  - Implementar exportação de dados de alunos (PDF ou Excel).
    Prazo: 3 dias
    Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

1.2. Testes

- **Testes unitários**
  - Validar cadastro de alunos fazendo inserção, edição e exclusão.
    Prazo: 1 dia
    Recursos Necessários: QA, desenvolvedor.

- **Testes de integração**
  - Testar a interação entre backend, banco de dados e frontend.
    Prazo: 1 dia
    Recursos Necessários: QA.

- **Testes de usabilidade**
  - Validar a experiência do usuário ao cadastrar alunos.
    Prazo: 1 dia
    Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema acadêmico.
    Prazo: 1 dia
    Recursos Necessários: DevOps.

- **Implantar sistema**
  - Subir o sistema em produção e realizar smoke tests.
    Prazo: 1 dia
    Recursos Necessários: DevOps, desenvolvedor.

Plano de Execução para Tela de Cadastro de Cursos

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados**
  - Criar tabelas para cursos.
    Prazo: 1 dia
    Recursos Necessários: Desenvolvedor backend, analista de banco de dados

- **Desenvolver API para gerenciamento de alunos**
  - Criar endpoints para registrar, editar, consultar e excluir informações de cursos.
    Prazo: 2 dias
    Recursos Necessários: Desenvolvedor backend.

- **Criar interface de cursos**
  - Desenvolver uma tela para listar cursos.
  - Inserir os botões: novo, editar, excluir e salvar.

- **Prazo**: 1 dia
  Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

1.2. Testes

- **Testes unitários**
  - Validar endpoints de cadastro de cursos.
    Prazo: 1 dia
    Recursos Necessários: QA, desenvolvedor.

- **Testes de integração**
  - Testar a interação entre backend, banco de dados e frontend.
    Prazo: 1 dia
    Recursos Necessários: QA.

- **Testes de usabilidade**
  - Validar a experiência do usuário ao cadastrar cursos.
  - **Prazo**: 1 dia
    Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema acadêmico.
    Prazo: 1 dia
    Recursos Necessários: DevOps.

- **Implantar sistema**
  - Subir o sistema em produção e realizar smoke tests.
    Prazo: 1 dia
    Recursos Necessários: DevOps, desenvolvedor.

Plano de Execução para Tela Emissão de Certificados

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados para emissão de certificados**
  - Criar tabelas para armazenar informações de certificados gerados, incluindo nome do aluno, curso, período, data de emissão e status (emitido, pendente).
  - Relacionar tabelas com os dados de alunos, turmas e cursos.
  - **Prazo:** 2 dias
  - **Recursos Necessários:** Desenvolvedor backend, analista de banco de dados.
- **Desenvolver API para emissão de certificados**
  - Criar endpoints para listar, gerar, consultar e reemitir certificados.
  - Implementar lógica para validação de critérios de emissão (frequência mínima, nota mínima).
  - **Prazo:** 3 dias
  - **Recursos Necessários:** Desenvolvedor backend.
- **Criar interface para emissão de certificados**
  - Desenvolver uma tela para seleção de turma e visualização de alunos aptos para receber certificado.
  - Adicionar funcionalidades para emitir certificados individuais ou em lote, com botões para exportar em PDF, enviar por e-mail ou salvar no sistema.
  - **Prazo:** 3 dias
  - **Recursos Necessários:** Desenvolvedor frontend, designer UI/UX.
- **Configurar envio automático de certificados**
  - Implementar integração com serviços de e-mail para envio automático de certificados aos alunos ou responsáveis.
  - **Prazo:** 2 dias
  - **Recursos Necessários:** Desenvolvedor backend.

1.2. Testes

- **Testes unitários**
  - Validar critérios de emissão e geração de certificados.
  - **Prazo:** 1 dia
  - **Recursos Necessários:** QA, desenvolvedor.
- **Testes de integração**
  - Garantir a comunicação entre backend, banco de dados e serviços de envio de e-mail.
  - **Prazo:** 1 dia
  - **Recursos Necessários:** QA.
- **Testes de usabilidade**
  - Validar a experiência do usuário na seleção de turmas e emissão de certificados.
  - **Prazo:** 1 dia
  - **Recursos Necessários:** QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema de emissão de certificados.
  - **Prazo:** 1 dia
  - **Recursos Necessários:** DevOps.
- **Implantar sistema**
  - Subir o sistema em produção e realizar smoke tests.
  - **Prazo:** 1 dia
  - **Recursos Necessários:** DevOps, desenvolvedor.

Plano de Execução para Tela Relatório de Aprovações

1. Tarefas Específicas do Projeto

1.1. Desenvolvimento

- **Planejar estrutura da base de dados para relatórios de aprovação**
  - Criar tabelas para consolidar dados de alunos aprovados e reprovados por turma, curso e período.
  - Relacionar tabelas com informações de frequência, notas e cursos.
  - Prazo: 2 dias
  - Recursos Necessários: Desenvolvedor backend, analista de banco de dados.
- **Desenvolver API para relatórios de aprovação**
  - Criar endpoints para consulta de dados de aprovação e geração de relatórios.
  - Implementar lógica para cálculo de taxas de aprovação e reprovação por turma, curso e período.
  - Prazo: 3 dias
  - Recursos Necessários: Desenvolvedor backend.
- **Criar interface para relatórios de aprovação**
  - Desenvolver uma tela com filtros (por curso, turma, período) e visualização dos dados de aprovação.
  - Adicionar funcionalidade para exportação dos relatórios em PDF e Excel.
  - Prazo: 3 dias
  - Recursos Necessários: Desenvolvedor frontend, designer UI/UX.

1.2. Testes

- **Testes unitários**
  - Validar cálculos de taxas de aprovação e exportação de relatórios.
  - Prazo: 1 dia
  - Recursos Necessários: QA, desenvolvedor.

- **Testes de integração**
  - Garantir a comunicação entre backend, banco de dados e frontend.
  - Prazo: 1 dia
  - Recursos Necessários: QA.
- **Testes de usabilidade**
  - Validar a experiência do usuário ao filtrar e visualizar relatórios de aprovação.
  - Prazo: 1 dia
  - Recursos Necessários: QA, designer UI/UX.

1.3. Implantação

- **Configurar ambiente de produção**
  - Preparar servidores e banco de dados para o sistema de relatórios de aprovação.
  - Prazo: 1 dia
  - Recursos Necessários: DevOps.
- **Implantar sistema**
  - Subir o sistema em produção e realizar smoke tests.
  - Prazo: 1 dia
  - Recursos Necessários: DevOps, desenvolvedor.

Cronograma de Atividades

Atividade	Início Previsto	Início Realizado	Final Previsto	Final Realizado	% Concluído	Recursos
Planejar estrutura da base de dados	16/11/2024		20/11/2024		0%	Desenvolvedor backend, analista de banco de dados
Desenvolver API para gerenciamento	18/11/2024		21/11/2024		0%	Desenvolvedor backend
Criar interface para frequência	22/11/2024		26/11/2024		0%	Desenvolvedor frontend, designer UI/UX
Implementar relatórios de frequência	27/11/2024		29/11/2024		0%	Desenvolvedor backend e frontend
Testes unitários	30/11/2024		04/12/2024		0%	QA, desenvolvedor
Testes de integração	05/12/2024		10/12/2024		0%	QA
Testes de usabilidade	10/12/2024		15/12/2024		0%	QA, designer UI/UX
Configurar ambiente de produção	16/12/2024		18/12/2024		0%	DevOps
Implantar sistema	19/12/2024		20/12/2024		0%	DevOps, desenvolvedor

Glossário

Metodologias de Execução

1. **Scrum**: Um framework ágil para gerenciamento de projetos, organizado em sprints (períodos curtos de trabalho, geralmente de 1 a 4 semanas), com entregas incrementais de valor.
2. **Kanban**: Um método visual de gerenciamento de fluxo de trabalho, que utiliza quadros e cartões para acompanhar tarefas em diferentes estágios.
3. **Lean**: Um modelo focado na eliminação de desperdícios e maximização de valor no processo de desenvolvimento.
4. **Sprints**: Intervalos de tempo fixos no Scrum, onde se desenvolvem e entregam partes do projeto.

Ferramentas de Execução

1. **Trello**: Uma ferramenta de gerenciamento de tarefas que utiliza quadros (boards) e cartões (cards) para organizar o trabalho.
2. **GitHub**: Um sistema de controle de versão e repositório para desenvolvimento colaborativo de código.

3. **Slack**: Uma plataforma de comunicação em equipe que centraliza mensagens, arquivos e integrações com outras ferramentas.
4. **Jenkins**: Uma ferramenta de automação de software usada principalmente para integração contínua (CI) e entrega contínua (CD).

Monitoramento e Controle

1. **Earned Value Management (EVM)**: Um método que mede o desempenho e o progresso do projeto com base em custo e cronograma.
2. **Critical Path Method (CPM)**: Uma técnica para determinar o caminho mais longo em um cronograma, identificando tarefas críticas.
3. **Program Evaluation and Review Technique (PERT)**: Uma técnica de análise que ajuda a estimar o tempo necessário para completar tarefas.
4. **Quality Assurance (QA):** É o processo de garantir a qualidade do produto de software, identificando bugs, falhas ou problemas antes do lançamento.

Termos de Desenvolvimento

1. **API (Application Programming Interface)**: Interface de programação que permite a comunicação entre sistemas ou componentes de software.
2. **Middleware**: Software que atua como um intermediário, facilitando a comunicação entre diferentes aplicativos ou sistemas.
3. **JWT (JSON Web Token)**: Um padrão para autenticação que transmite informações seguras entre cliente e servidor por meio de tokens.

Ambiente e Infraestrutura

1. **DevOps**: Um conjunto de práticas que integra desenvolvimento (Dev) e operações (Ops) para entregar software com maior eficiência.
2. **Smoke Test**: Um teste inicial rápido para verificar se o sistema principal está funcionando corretamente.